

Module:7

AWS Application Integration & Architectural Best Practices

- AWS Application Integration, Amazon SQS (Simple Queue Service), Amazon SNS (Simple Notification Service), Amazon SWF (Simple Workflow Service), AWS Step Functions, AWS Well-Architected Framework, Design Principles for AWS Cloud Architectures, Scalability and Elasticity, High Availability and Fault Tolerance, Security, Cost Optimization.

- **AWS Application Integration** services help connect and coordinate distributed applications, enabling seamless communication and automation between systems without complex custom code

some key services under AWS Application Integration:

- **Amazon Simple Queue Service (SQS):** A fully managed message queuing service that enables you to decouple and scale microservices, distributed systems, and serverless applications. It allows messages to be sent between components of an application asynchronously.

Ex. **Amazon SQS:** An order processing system where orders are queued for inventory checks.

- .

- **Amazon Simple Notification Service (SNS):** A managed service that provides **Publish/Subscribe) messaging** for sending notifications. It enables application-to-application (A2A) and application-to-person (A2P) communication.
- **Ex. Amazon SNS:** Sending notifications to users via SMS or email when their order status changes.

Simple Workflow Service

- Amazon SWF (Simple Workflow Service) is a fully managed service that helps **developers build, run, and scale complex, long-running workflows**. It coordinates tasks across distributed applications, ensuring **reliable execution and tracking of each step**.
- Ex. Amazon SWF can be used to manage an order fulfillment process, coordinating tasks like payment processing, inventory check, and shipping across multiple systems.

- **Amazon EventBridge:** A serverless event bus that makes it easy to connect applications using events. It allows you to route real-time data from one application to another in a highly scalable manner.
- **Ex.Amazon EventBridge:** Triggering a billing system when a user completes a purchase.

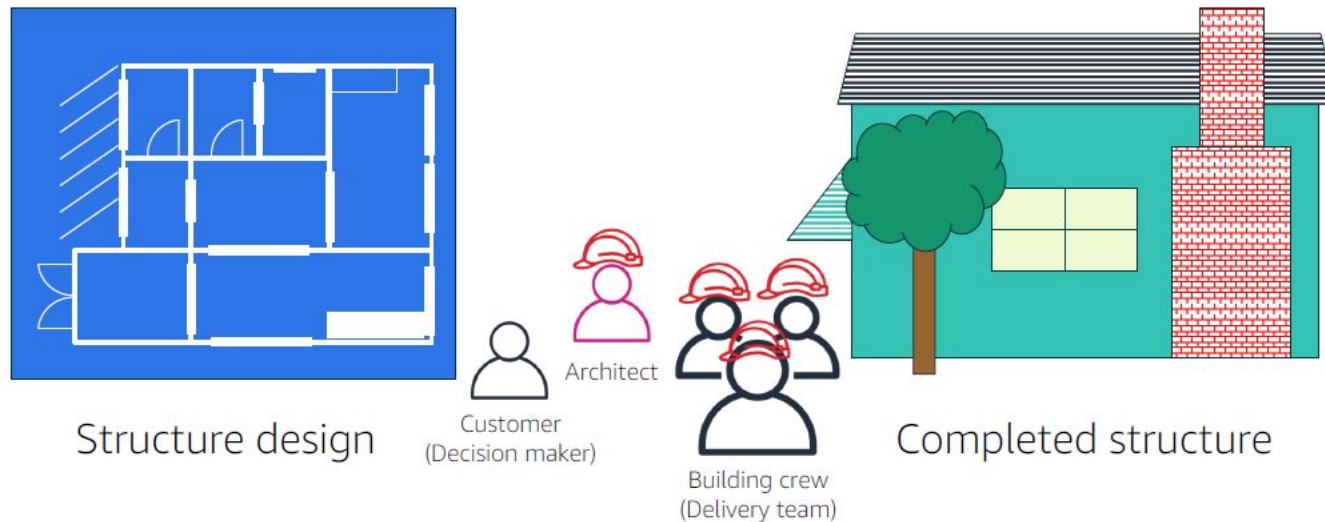
- **AWS Step Functions:** A serverless orchestration service that allows you to coordinate multiple AWS services into a single workflow, helping to automate business processes and manage the dependencies between different tasks.
- **Ex.AWS Step Functions:** Automating a multi-step loan approval process across different systems.

- **Amazon AppFlow:** A service that helps automate data flows between AWS services and other SaaS (Software as a Service) applications without the need for custom integrations.
- **Ex.Amazon AppFlow:** Syncing customer data between Salesforce and an AWS S3 bucket.

AWS Well-Architected Framework

- **AWS Well-Architected Framework** is a set of best practices and guidelines designed to help cloud architects build secure, high-performing, resilient, and efficient infrastructure on AWS. It's organized around **six pillars** to ensure workloads meet business requirements effectively:

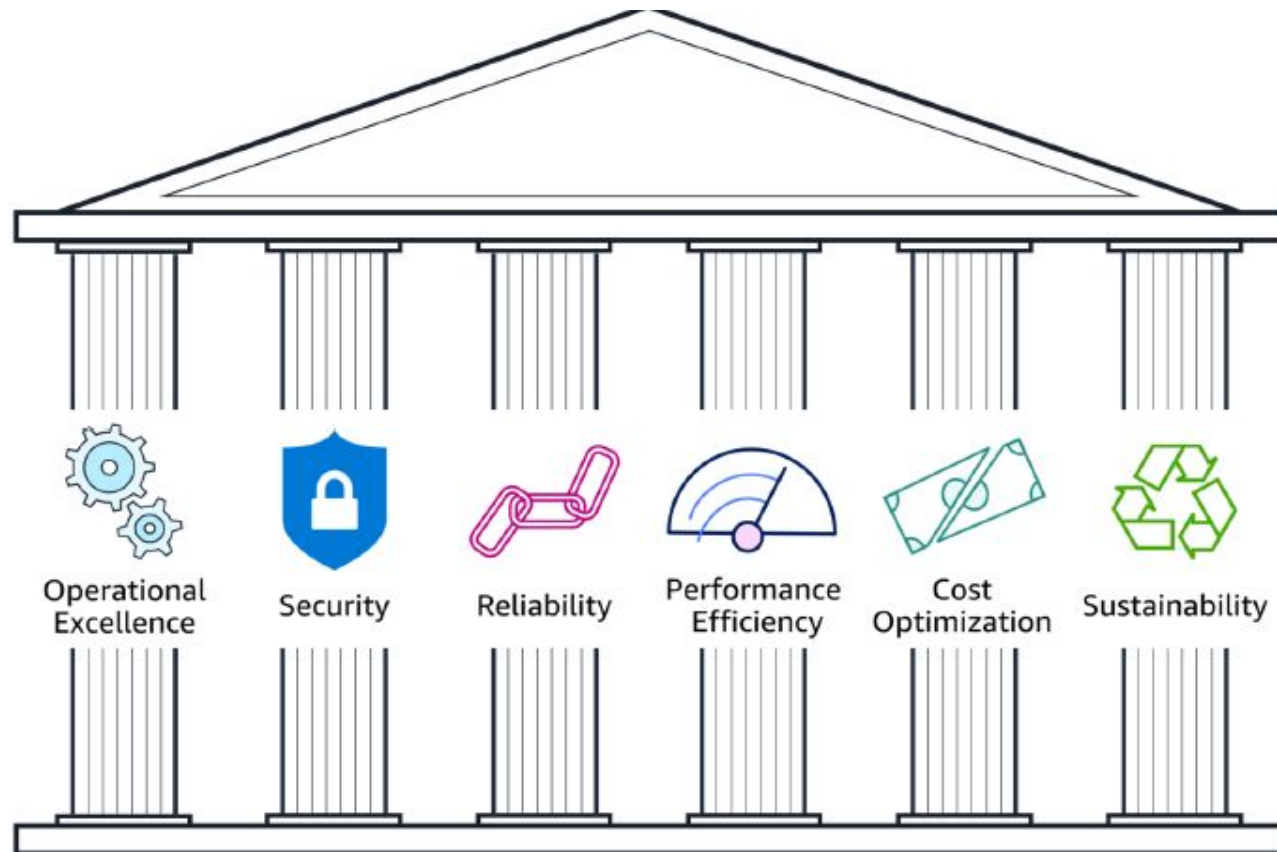
Architecture: designing and building



What is the AWS Well-Architected Framework?

- A guide for designing infrastructures that are:
 - Secure
 - High-performing
 - Resilient
 - Efficient
- A consistent approach to evaluating and implementing cloud architectures
- A way to provide best practices that were developed through lessons learned by reviewing customer architectures

Pillars of the AWS Well-Architected Framework



- Operational Excellence
- Security
- Reliability
- Performance Efficiency
- Cost Optimization
- Sustainability

- **Operational Excellence:** Automate code deployments with AWS CodePipeline to ensure consistent, reliable updates.
- **Security:** Use AWS Identity and Access Management (IAM) roles to restrict access to critical resources.

- **Reliability:** Deploy databases with Amazon RDS Multi-AZ to ensure automatic failover in case of outages.
- **Performance Efficiency:** Use Amazon CloudFront for fast, low-latency content delivery across global locations

- **Cost Optimization:** Use Auto Scaling to automatically adjust resources based on demand, minimizing costs.
- **Sustainability:** Utilize AWS Graviton processors to improve energy efficiency and reduce carbon footprint

Scalability and Elasticity

- **Horizontal Scaling:** Add more instances or nodes instead of upgrading the capacity of individual components. Use services like **Auto Scaling Groups** to handle changes in demand automatically.
- **Stateless Architectures:** Design components to be stateless or share state through distributed stores, like Amazon **DynamoDB** or **ElastiCache**, to easily scale horizontally.
- **Event-Driven Scaling:** Use **AWS Lambda** for on-demand execution or services like **Amazon Kinesis** to scale processing capacity based on real-time events.
- **Decoupling Components:** Use managed services like **Amazon SQS** and **Amazon SNS** to separate components, allowing independent scaling.

High Availability and Fault Tolerance

- **Multi-AZ Deployment:** Distribute critical workloads across multiple Availability Zones (AZs) to minimize the impact of individual zone failures. Services like **Amazon RDS** support Multi-AZ for database redundancy.
- **Cross-Region Replication:** For global applications, use **Amazon S3 Cross-Region Replication** and **Amazon Aurora Global Database** for low-latency access and fault tolerance across regions.
- **Load Balancing and Health Checks:** Use **Elastic Load Balancing (ELB)** to distribute traffic across instances, with health checks to route traffic only to healthy targets.
- **Graceful Failover:** Set up automated failover with **Route 53** for DNS failover and services like **Elastic IP** for addressing redundancy.

Security

- **Principle of Least Privilege:** Grant the minimum permissions necessary for each component and user. Utilize **AWS Identity and Access Management (IAM)** to control access.
- **Data Encryption:** Encrypt data in transit and at rest using AWS Key Management Service (KMS) and **AWS Certificate Manager (ACM)** for SSL/TLS certificates.
- **Network Isolation:** Use **Amazon VPC** to create isolated network environments, with security groups, Network ACLs, and private subnets to restrict access.
- **Automated Threat Detection:** Implement monitoring and automated threat detection using **AWS CloudTrail** for logging, **Amazon GuardDuty** for anomaly detection, and **AWS Security Hub** for consolidated findings

Cost Optimization

- **Right-Sizing Instances:** Match instance types and sizes to workload requirements. Use **Compute Optimizer** and **Trusted Advisor** to analyze and recommend optimal configurations.
- **Spot Instances and Reserved Instances:** Utilize **Spot Instances** for non-critical workloads and **Reserved Instances** or **Savings Plans** for predictable workloads.
- **Auto Scaling:** Use Auto Scaling to add or remove resources automatically based on demand, avoiding the cost of over-provisioning.
- **Storage Optimization:** Choose cost-effective storage options like **S3 Infrequent Access** for infrequently accessed data and **S3 Intelligent-Tiering** to optimize storage costs over time.
- **Serverless Architectures:** Leverage **AWS Lambda** and other serverless services like **DynamoDB** and **API Gateway** for pay-as-you-go pricing and minimized infrastructure management.